

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2.SymbolTableandMemory Allocation

Given the following symbol table entries:

VariableData TypeSize(Bytes)

p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

```
if (x > y || p < q)
```

Intermediate code:

```
if x > y goto L1
```

```
if p < q goto L1
```

```
goto L2
```

```
L1: ...
```

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5.Backpatching and Flow Control

Given:

Q1. Intermediate Code Analysis and Optimization

The given three-address code is:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

The first instruction calculates the addition of x and y and stores the result in t1. The second instruction takes the value stored in t1 and multiplies it by z, giving the result in t2. The third instruction divides t2 by w and stores the result in t3. Finally, the fourth instruction subtracts p from t3 and stores the final result in t4.

Therefore, the equivalent high-level expression is $((x + y) \times z / w) - p$. The operations are performed from the inner expression towards the final subtraction.

The equivalent expression tree can be represented as follows:

```
(-)
 / \
 (/) p
 / \
 (*) w
 / \
 (+) z
 / \ x
 y
```

Here, the root node is the subtraction operator. Its left child is the division operation and its right child is p. The division operation has multiplication as its left child and w as its right child. The multiplication has the addition operation as its left child and z as its right child. Finally, x and y are the operands of the addition operation.

Temporary variables are very useful in intermediate code generation. A complex expression is divided into smaller three-address instructions, and every temporary stores one intermediate result. This makes the code easier for the compiler to analyze and translate into machine instructions.

Temporary variables also help optimization. If an intermediate result is required again, the compiler can reuse the stored value instead of calculating it again. They also make register allocation easier because the compiler can determine when a temporary is created and when its value is no longer needed.

Another advantage is that the evaluation order becomes clear. In this example, t1 must be calculated before t2, t2 before t3, and t3 before t4. Therefore, the sequence can be easily converted into target-machine instructions.

Explanation: Temporary variables divide a complicated expression into simple steps, make the evaluation order clear, and provide useful intermediate values for optimization and code generation.

Q2. Symbol Table and Memory Allocation

The given symbol table contains the following variables:

Variable	Data	Type	Size
p	char	1 byte	q double
8 bytes	r	int	4 bytes
s	float	4 bytes	

The compiler uses the symbol table to keep information about identifiers declared in a program. In this case, the table contains the variable name, its data type and the amount of memory normally required for that type.

The raw memory requirement is calculated by adding the sizes of all four variables:

```

p = 1 byte
q = 8 bytes
r = 4 bytes
s = 4 bytes

Total = 1 + 8 + 4 + 4
       = 17 bytes

```

Therefore, the total memory required without considering padding or alignment is **17 bytes**. The compiler can assign an address or offset to each variable and use the information in the symbol table while generating machine or intermediate code.

Data types affect memory organization because different types have different sizes and representations. A char normally needs only one byte, while a double requires more storage. An int and a float are both listed as 4

bytes in the given table, but their representations and operations are different.

Alignment is another important factor. Some processors require certain data types to start at particular memory boundaries. For example, a double may need an address aligned to an 8-byte boundary. If a char is placed immediately before a double, the compiler may insert unused padding bytes so that the double starts at the correct boundary.

For this reason, the calculated 17 bytes is the **raw storage requirement**. The actual memory occupied by a structure or memory block can be larger when alignment padding is added. The exact padded size cannot be calculated from the question alone because the alignment rules of the target processor are not specified.

Good memory organization is important because it affects both memory usage and execution efficiency. Correct alignment can allow the processor to access data efficiently, while poor organization may increase memory consumption or cause slower accesses on some architectures.

Explanation: The four variables need 17 bytes in total before padding, while data-type alignment can increase the actual memory allocated by the compiler.

Q3. Optimization in Intermediate Code

The given intermediate code is:

```

t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3

```

The execution starts with the calculation of $m \times n$, and the result is stored in t1. Next, $p + q$ is calculated and stored in t2. After that, the compiler calculates $m \times n$ again and stores the same result in t3. Finally, $t2 - t3$ is calculated and stored in t4.

The important observation is that the expression $m \times n$ occurs twice. Assuming the values of m and n have not changed between the two statements, both calculations produce exactly the same result.

Therefore, the original code performs one unnecessary multiplication. This is a suitable example of **common subexpression elimination**. The optimized code can be written as:

```

t1 = m * n
t2 = p + q
t3 = t1 t4 =
t2 - t3

```

The compiler can go one step further if the temporary t3 is not needed anywhere else. The optimized code becomes:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

Now the multiplication $m \times n$ is performed only once. This reduces the number of arithmetic instructions and therefore reduces the amount of work performed by the processor.

This optimization also reduces the need for additional temporary storage. Fewer instructions can lead to lower execution time and, depending on the target system, lower energy consumption. Such improvements become important when the same code is executed many times, such as inside loops or during large-scale data processing.

The compiler must be careful before applying this optimization. It should verify that m and n have not changed and that the expression has no side effects. Only when reusing the result is semantically safe can the common subexpression be eliminated.

Thus, the main optimization opportunity in the given code is to remove the repeated calculation of $m \times n$ and reuse $t1$.

Explanation: Common subexpression elimination avoids calculating $m \times n$ twice, reducing the number of instructions and improving execution efficiency.

Q4. Control Flow and Boolean Expression Handling

The Boolean expression given is: **if ($x > y \parallel p < q$)**. The intermediate code is:

```
if x > y goto L1
if p < q goto L1
goto L2 L1: ...
```

The expression uses the logical OR operator. For an OR operation, the complete expression is true when at least one of the two conditions is true.

First, the compiler checks the condition $x > y$. If x is greater than y , the result of the whole OR expression is already known to be true. Therefore, the compiler immediately jumps to L1.

If $x > y$ is false, the compiler must check the second condition $p < q$. If p is less than q , the expression is true and the compiler jumps to L1. If $p < q$ is also false, both conditions are false, so the OR expression is false and execution proceeds to L2.

The control flow can therefore be understood as:

```
x > y ? /
\ true
false | |
L1 p < q ?
/ \ true
false
| |
L1 L2
```

This is called **short-circuit evaluation**. The second condition is evaluated only when it is necessary. This can reduce execution time because the compiler does not evaluate conditions that cannot affect the final result.

Short-circuit evaluation can also be useful when the second expression is expensive to calculate. For example, if the second condition involved a function call or memory access, it would be avoided when $x > y$ is already true.

The intermediate code therefore provides an efficient representation of the Boolean expression using conditional jumps rather than first calculating a separate Boolean value.

Explanation: Short-circuit evaluation checks the second condition only when the first condition is false, thereby avoiding unnecessary work.

Q5. Backpatching and Flow Control

The given instructions are:

```
Instruction Target
200: if x == y goto _
201: goto _

truelist      =      {200}
falselist = {201}
```

The blank targets show that the compiler has generated jump instructions but does not yet know the final destination addresses. This situation is common when translating Boolean expressions and control-flow statements because the destination code may be generated later.

Instruction 200 belongs to the **truelist**. This means that when $x == y$ is true, instruction 200 must jump to the code that handles the true result. Instruction 201 belongs to the **falselist**, so it must jump to the code that handles the false result.

Suppose the compiler later generates two labels called **Ltrue** and **Lfalse**. The backpatching operation fills the missing targets as follows:

```
200: if x == y goto Ltrue
201: goto Lfalse
```

If the compiler later knows the actual instruction addresses, the labels can be replaced by those addresses. For example, if **Ltrue** corresponds to address 250 and **Lfalse** corresponds to address 270, the instructions could become:

```
200: if x == y goto 250
201: goto 270
```

The exact addresses depend on the code generated after these instructions, so the important point is the process rather than particular numerical values.

Backpatching is useful because the compiler does not need to know every target address at the moment it generates the conditional jump. It can keep a list of instructions whose targets are missing and fill them later when the required labels are available.

In Boolean expression translation, true-list and false-list are commonly maintained. The true-list contains jumps that should be patched to the true destination, while the false-list contains jumps that should be patched to the false destination.

Backpatching is also useful in if-else statements, while loops, for loops and other control-flow constructs involving forward jumps. It makes intermediate-code generation simpler and supports systematic translation of complex Boolean expressions.

Therefore, backpatching connects incomplete jump instructions with their final control-flow destinations and is an important technique in intermediate-code generation.

Explanation: Backpatching allows the compiler to generate incomplete jumps first and fill their correct target labels or addresses after the destination code is known.

Overall conclusion: The problems demonstrate important intermediate-code concepts. Three-address code represents complex expressions using simple instructions and temporaries. Symbol tables help the compiler organize variables and memory requirements. Optimization removes redundant computations such as repeated common subexpressions. Boolean expressions can be translated efficiently using short-circuit control flow, while backpatching resolves forward jumps whose destinations are not known immediately. Together, these techniques help a compiler produce correct, efficient and organized intermediate code.

Instruction Target

200: ifx==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications Presents	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

